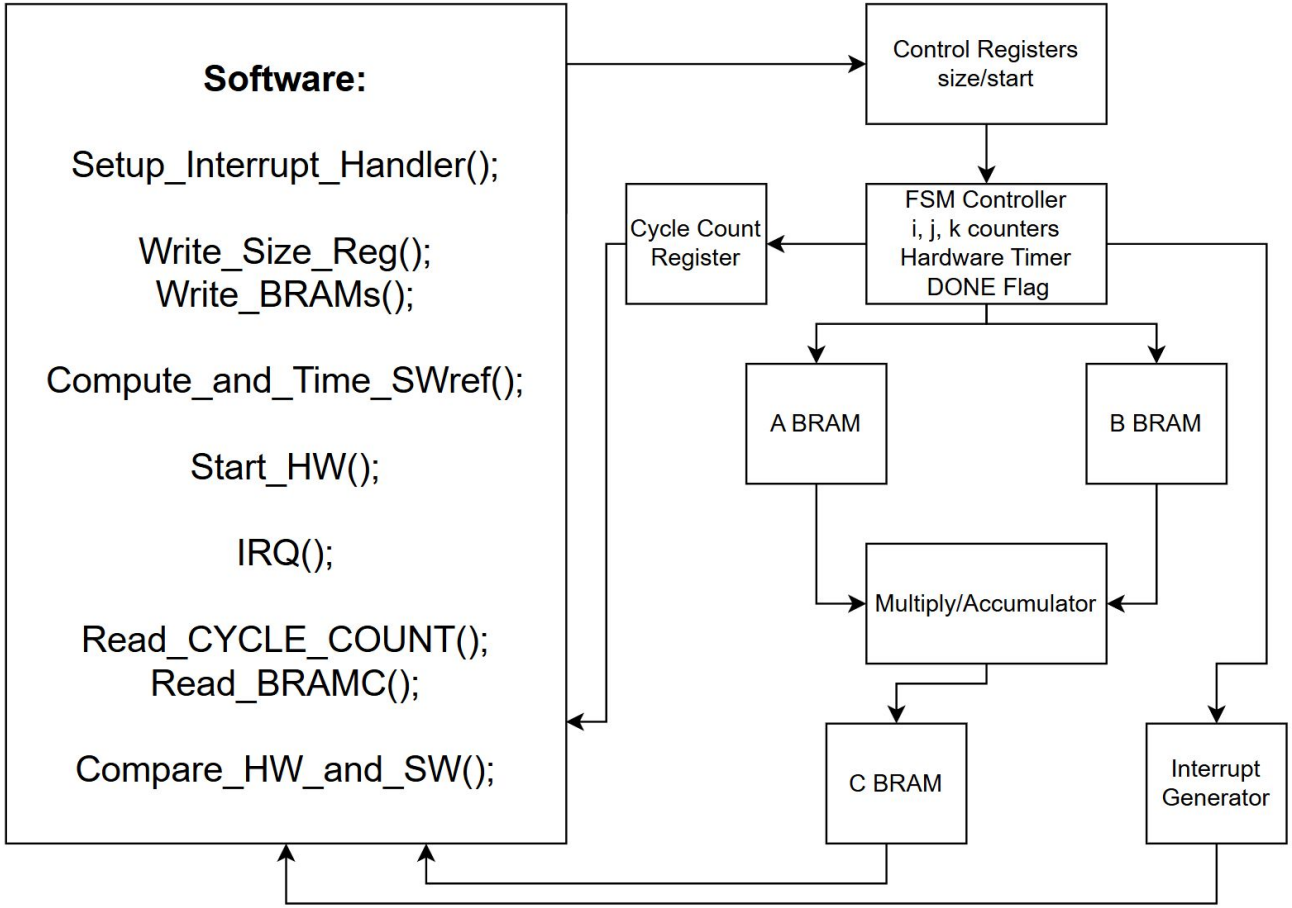
# Matrix Multiplication Accelerator

California State University Northridge
ECE 520
Daniel Celnik

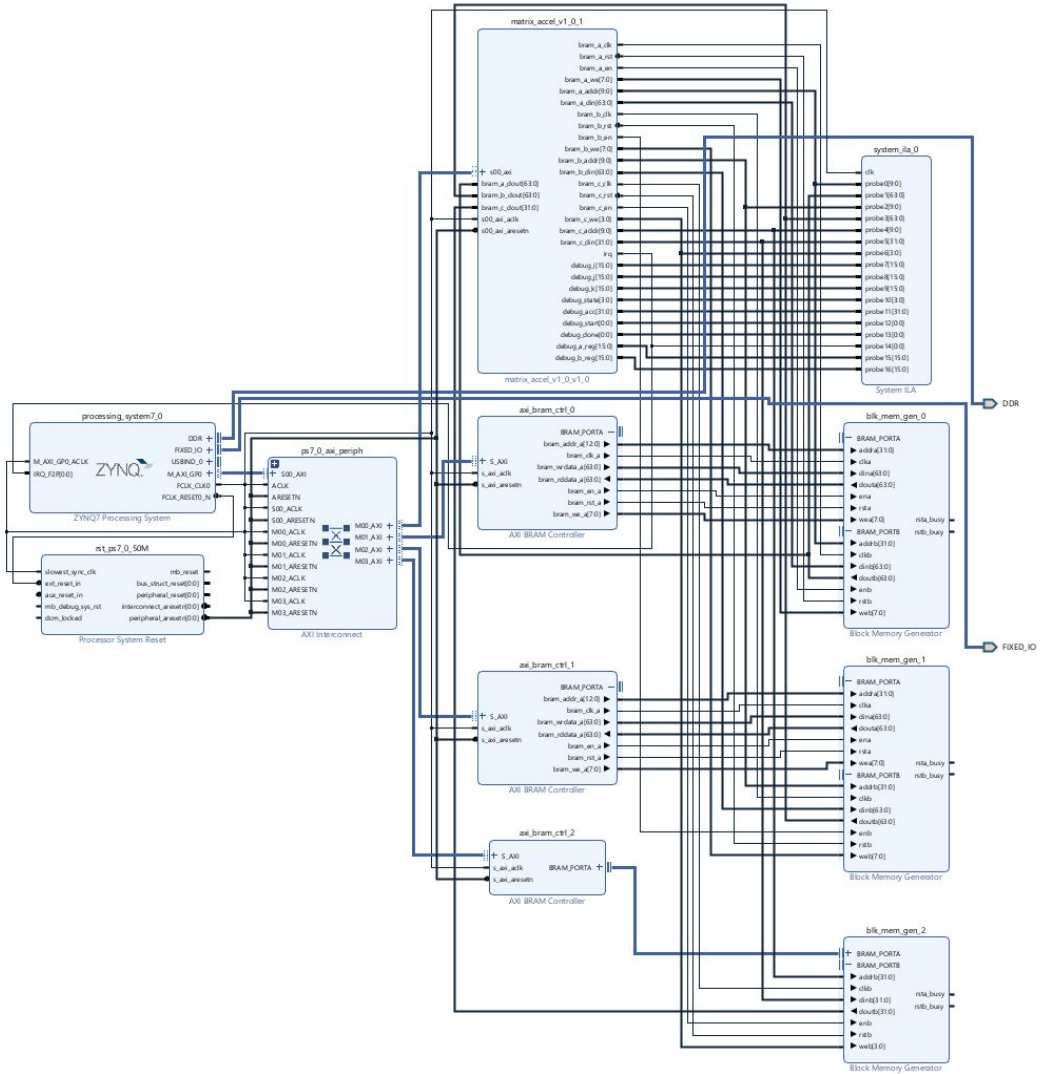
# System Overview

- Employs matrix operation: A x B = C
- PS configures AXI-Lite registers (size, int_en, reset)
- PS writes Matrices A/B to BRAM
- PS computes a software reference for Matrix C
- PS writes the START_bit_mask to AXI-Lite Control register to start the Accelerator
- PL Hardware execution using MAC architecture
- PL writes the Matrix C result to BRAM
- PS reads from BRAM C and compares HW and SW results

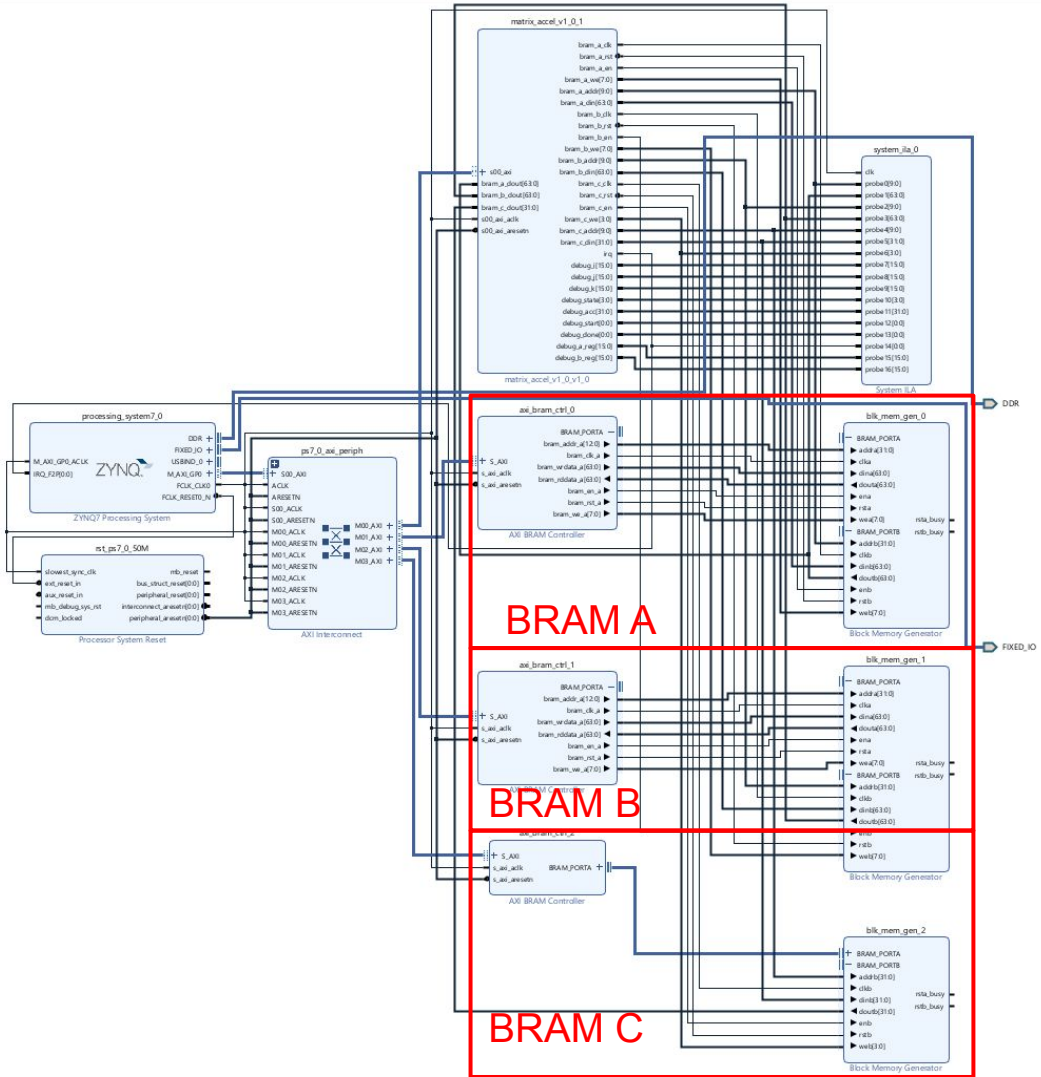
# Block Diagram



# Final Vivado Block Design



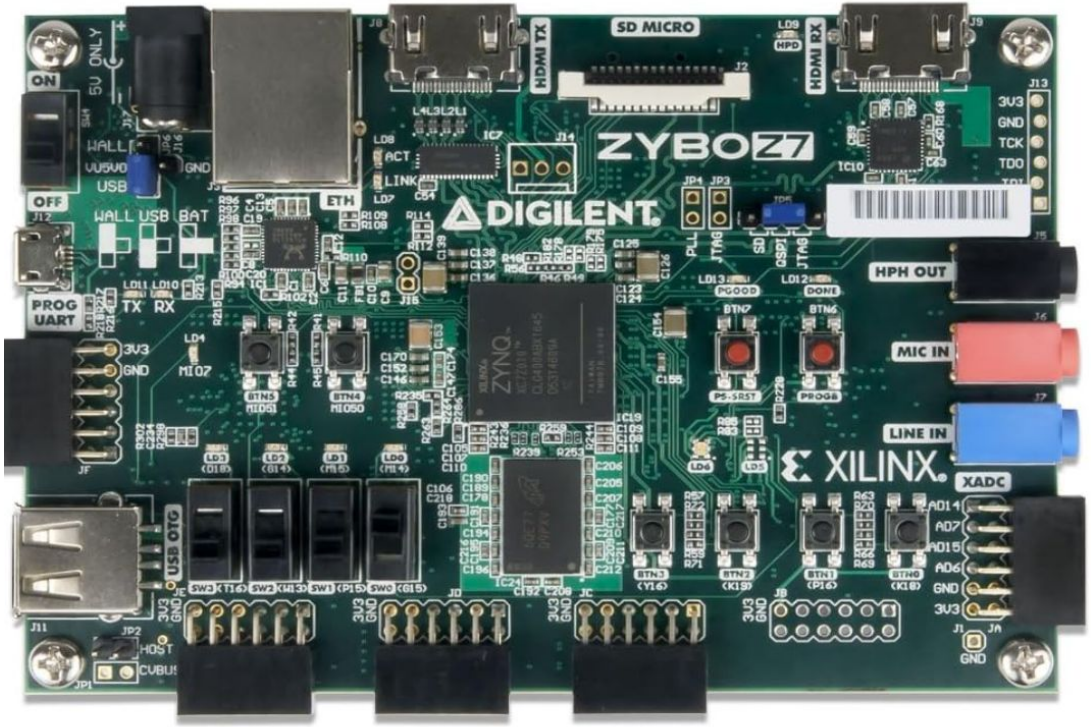
# Final Vivado Block Design



# Components Used

- Zybo Z7 ARM/FPGA SoC Board

- USB-A to USB Micro-B Cable



# AXI-Lite Control Registers

| Offset | Name | Description |
|---|---|---|
| 0x00 | CONTROL | Start, interrupt enable, soft reset |
| 0x04 | SIZE | Matrix dimension (N) |
| 0x08 | CYCLE_COUNT | Execution cycle count |
| 0x0C | STATUS | Busy/done flags, FSM state |

# Matrix Multiplication Conceptual Background

[row][column]

$$\boxed{C[i][j] = \sum_{k=0}^{N-1} (A[i][k] * B[k][j])}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$= \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \\ a_{31}b_{11} + a_{32}b_{21} & a_{31}b_{12} + a_{32}b_{22} \end{bmatrix}$$

# BRAM Address Generation (1-D)

1. Address_A_Index = i*N + k
2. Address_B_Index = j*N + k
3. Address_C_Index = i*N + j

Where:

i = row index

j = column index

N = Size of matrix

# Matrix Calculation

- Once initial BRAM A/B addresses are calculated, elements are latched
  - A_reg and B_reg
- Accumulator is incremented by A_reg * B_reg
- New BRAM A/B addresses are calculated (k increments)
- **Repeat until k = N-1**
- Accumulator now holds the final value for C[0][0]
- Write result to BRAM C and increment Matrix C's column index (j)
- **Repeat until Matrix C's first row is filled (j = N-1)**
- Increment Matrix C's row index (i)
- **Repeat until Matrix C is full (i = N-1)**

# State Selection/Sequential Logic

1. ST_IDLE (default) - waits for the PS to write to the START register
2. ST_CLEAR_ACC
3. ST_READ_AB (present addresses)
4. ST_WAIT_AB1 (1-cycle read latency)
5. ST_WAIT_AB2 (1-cycle settling time)
6. ST_LATCH_AB
7. ST_MAC
8. ST_CHECK_K - returns to ST_READ_AB if (k < N-1)
9. ST_WRITE_C
10. ST_CHECK_J - returns to ST_CLEAR_ACC if (j < N-1)
11. ST_CHECK_I - returns to ST_CLEAR_ACC if (i < N-1)
12. ST_DONE

# Block Memory Generator Interface

| Signal Name | Description | Custom IP Implementation |
|---|---|---|
| clk | Clock signal | Tied to PS's axi_aclk |
| rst | Reset signal | Tied to 0; not used. The BRAM units always hold the last written data |
| en | Enable signal for BRAM access | Asserted when the accelerator performs a read or a write; deasserted when idle |
| we | Write enable | Determines read vs write; '0' = read, '1' = write |
| addr | Address bus | Specifies which memory location to access |
| din | Data input bus | Carries data from accelerator |
| dout | Data output bus | Carries data to accelerator |

# Software: Menu User Interface

1. Load default matrices into software buffers.
2. Write matrices A and B into BRAM.
3. Compute the software reference (timed).
4. Run the hardware accelerator using interrupts.
5. Read matrix C from BRAM and compare HW/SW results.
6. Print accelerator registers.
7. Print matrices A, B, C_hw, and C_sw.
8. Enable interrupts.
9. Disable interrupts.
10. Run the full matrix test using interrupts (1-5).
11. Assert the hardware accelerator’s soft reset.
12. Change the matrices’ sizes.
13. Print hardware control registers’ base addresses and interrupt information.
14. Display this menu.

# Throughput Scaling via Parallelism

- SW and HW calculation methods are equivalent; how do we leverage PL resources to increase throughput?

Increasing RTL complexity is easy; we need to send more data at once!

- **Each Matrix A/B element is an int16**
- 1-MAC: 16-bit data bus
- 2-MAC: 32-bit data bus
- 4-MAC: 64-bit data bus
- **Each Matrix C element is an int32**

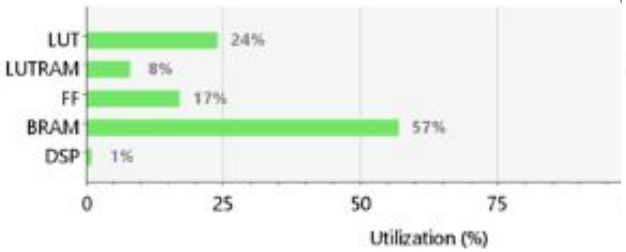
# Throughput Scaling via Parallelism cont. (4-MAC)

- Need a packing helper function in SW to pack 4 int16s into an int64

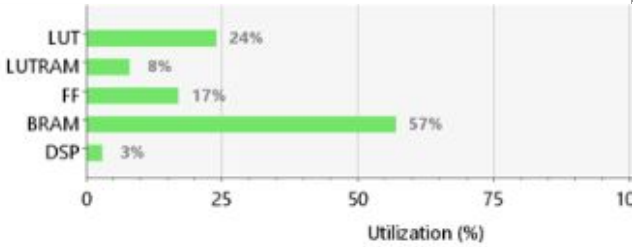
- Need bit-slicing registers in HW to extract 4 int16s from an int64

# Results: Resource Utilization

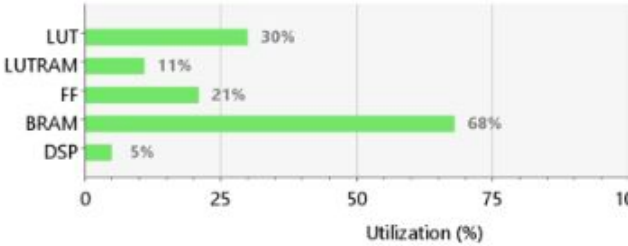
## 1-MAC



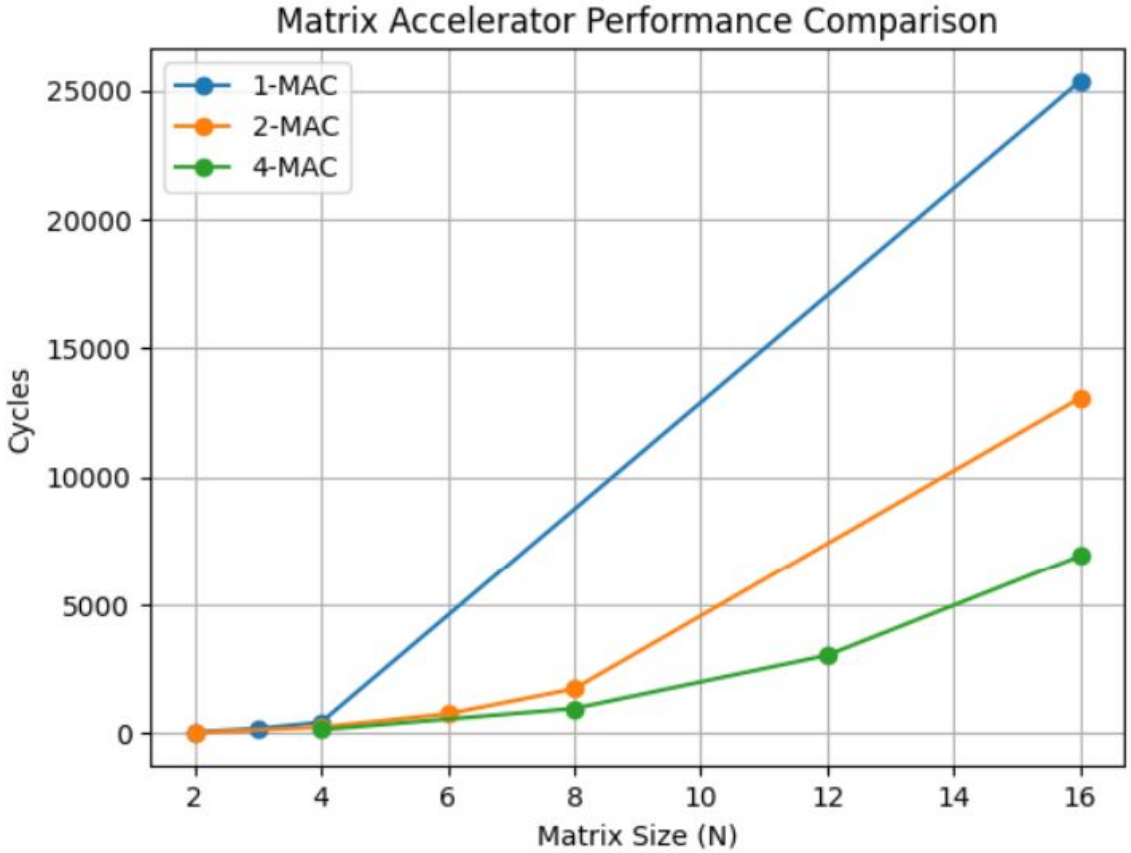

## 2-MAC



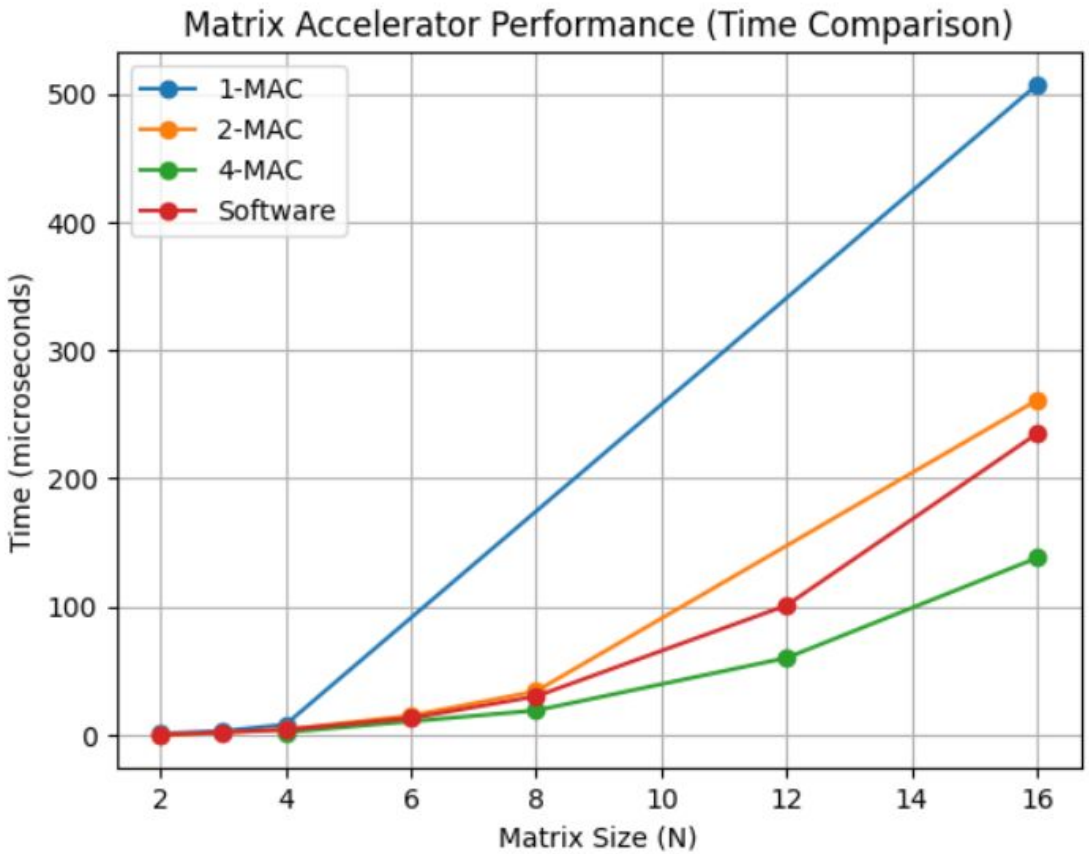

## 4-MAC



# Results: Performance Scaling



# Results: HW vs SW Compute Time



# Demonstration Video

https://youtu.be/MQN0aFwF9Ak

# AI Tool Usage

- Hardware:
  - Guide implementations
  - Write testbenches for matrix accelerator IP prototypes
  - Debug incorrect functionality and IP updating issues
- Software (functions):
  - Enhance user input by allowing multiple input characters via UART buffer helper function
  - Scale BRAM write functions to 2-MAC and 4-MAC

- Increasing productivity without decreasing conceptual understanding!

# Conclusion

This project explored:

- AXI4 handshaking interfaces
- PS-PL communication techniques
- PL-based parallelism scaling
- BRAM-based memory architecture
- Finite state machine control design
- AXI-Lite register designation
- Hardware-software co-design for performance optimization and verification

# References

[1] AMD Xilinx, *AXI Reference Guide*, UG1037.

[2] AMD Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[3] AMD Xilinx, *Block Memory Generator v8.4 Product Guide*, PG058.

[4] AMD Xilinx, *AXI BRAM Controller v4 Product Guide*, PG078.

[5] AMD Xilinx, *Vivado Design Suite User Guide: Designing with IP*, UG896.